

Comparative Analysis of State Representations for Deep Q-Learning in Snake Game

Problem Statement

Playing Snake effectively requires strategic planning that traditional programming struggles with. While reinforcement learning offers promise, the choice of state representation significantly impacts learning efficiency and final performance. This project investigates which state representation yields the most effective and sample-efficient learning for Snake.

Research Questions

1. RQ1(Primary): How do different state representations (feature-based vs.CNN-based) affect the performance and training efficiency of DQN agents in Snake?
2. RQ2(Baseline): How does DQN performance compare to human baseline performance?
3. RQ3(Progression): What qualitative strategies emerge at different training stages, and how do they differ between representations?

Methodology: Experimental Comparative Study

Research Design: A controlled experiment comparing two DQN variants with different state representations against established baselines.

Compared Agents:

1. Human Baseline: Average performance from 5 human players (10 games each)
2. Random Agent: Stochastic policy (control baseline)
3. DQN-Feature: Dense neural network processing engineered features
4. DQN-CNN: Convolutional neural network processing grid images

State Representations:

- Feature-based (DQN-Feature): Vector of 11 features:

text

[food_dx, food_dy, # Relative food position (normalized) danger_left, danger_front, danger_right, # Binary collision sensors direction_left, direction_right, direction_up, direction_down, # One-hot snake_length_normalized] # Length / max_possible_length

- CNN-based (DQN-CNN): 30×30×3 grid where:
 - Channel 0: Snake body (1.0 for body, 0.0 otherwise)

- Channel 1: Snake head (1.0 for head)
- Channel 2: Food position (1.0 for food)

Training Protocol:

- Computational Budget: 10,000 training episodes per agent (feasible on CPU)
- Evaluation: 100 episodes every 1,000 training episodes ($\epsilon=0$)
- Random Seeds: 3 different seeds for statistical robustness
- Hardware Assumption: Standard laptop CPU (no GPU acceleration)

Justified Hyperparameters:

- Batch Size: 64 (common for DQN, balances stability and efficiency)
- Replay Buffer: 10,000 experiences (standard for medium-sized problems)
- Target Update: Every 100 episodes (moderate stability-memory trade-off)
- Learning Rate: 0.001 (Adam default, works well for many RL problems)
- γ (Discount): 0.95 (values short-term rewards while considering medium-term)
- ϵ -decay: 1.0 $\rightarrow$ 0.1 over first 5,000 episodes, then fixed (standard schedule)

Metrics:

1. Primary: Mean score over 100 evaluation episodes 2. Secondary:

- Training efficiency: Episodes to reach human baseline performance
- Maximum score achieved
- Survival time (moves per game)
- Qualitative strategy analysis (via gameplay recordings)

Human Baseline Protocol:

- Recruit 5 participants with basic Snake experience
- Each plays 10 games with the same interface agents use
- Record average and distribution of scores
- This provides a realistic performance target and interpretable benchmark

Expected Results

1. Performance Hierarchy: DQN - CNN > DQN - Feature > Random, with DQN agents approaching but not exceeding expert human performance within the training budget.
2. Training Efficiency: DQN - Feature will learn faster initially but plateau earlier; DQN-CNN will have slower start but higher final performance.
3. Human Comparison: We expect DQN agents to surpass average human performance but remain below expert human play due to limited training budget.
4. Emergent Behaviors: DQN-CNN will develop more spatially-aware strategies; DQN-Feature will exhibit more predictable, rule-like behaviors.

Contingency Plans

1. If training is too slow: Reduce gridsize to 20×20 for faster convergence while maintaining problem complexity
2. If DQN struggles: Implement simpler Double DQN instead of vanilla DQN
3. If human baselining is difficult: Use pre-recorded human game play data or

simplify to single experienced player

RQ1

The snake game itself is code generated by Henrik Strøm. The code is written in python and is using PyGame for rendering and control.

In order to wrap the Snake game into a clean Reinforcement Learning environment, without rewriting the code and without adding unnecessary code, to create a sort of remote control for the existing game. Meaning we are not actually changing the game rules, but we are removing the keyboard input, taking control of the snake programmatically and running the game one step at a time.

The original game is located in the folder `environment/snake_game.py` and is kept mostly unchanged, the classes `Vektor`, `Snake` and `Food` has not been modified, but we do have had to adapt the `SnakeGame` class slightly. The minimal changes to `SnakeGame`, has been done because the class has a `run()` method which contains a while-loop and keyboard input, which we do not want in RL, so instead we will ignore it and drive the game manually from the class `SnakeEnvironment`, located in the folder `environment/snake_env.py`, keeping the code more or less traceable and improved.

Environment wrapper

The class `SnakeEnvironment` starts with the minimal skeleton in the `__init__` method, this makes it possible for the wrapper to own the game, the snake and the food and tracks termination. In order to start a new episode, we implemented the `reset()` method, the key idea here, is when a new episode started it has a fresh snake, fresh food, not score carryover and returns the initial state. The private method `_apply_action()` has the function of replacing keyboard input with actions, so instead of keyboard it handles integer action.

The entire RL environment logic lies in the implementation of `step(action)`, this advances the game by exactly one timestep. In the beginning of the project we did not need a real state in order to test if the wrapper worked, therefore we used a state placeholder for now, which will be replaced with feature-based state and CNN grid state.

In order to know if the environment is correct, we needed to test with a random Agent, we created a quick test, in order to check if the snake moves, food appeared, game ends and no crashes happens. The test worked and we could then be sure that agents is allowed to

interact with the game without modifying the underlying game logic.

```
# Test with a random agent
env = SnakeEnvironment(render=True)
state = env.reset()

done = False
while not done:
    action = np.random.randint(0, 4)
    state, reward, done = env.step(action)

print("Episode finished")
```

Actions and rewards

When defining the actions and rewards, we only did it once and have to keep it forever. It is a very important step, because when we lock actions and rewards in, we don't change them again, which is what will later make our comparison fair. The goal of defining the actions and rewards, is that every agent (Random, DQN-feature, DQN-CNN or Human comparison) interacts with exactly the same action space and reward function. We used a sparse reward function, providing +1 for eating food, -1 for terminal collisions and 0 otherwise. This minimal reward design avoids excessive reward shaping and ensures that differences in performance arise primarily from state representation rather than handcrafted incentives.

Implementation of feature-based state representation